Objetivo de la lección

Explorar la importancia de manejar excepciones específicas para mejorar la claridad, la precisión y la calidad del código.

Repaso breve de la Jerarquía de Excepciones en Python.

Antes de profundizar en el manejo específico de excepciones, es crucial entender la jerarquía de excepciones en Python. Esto proporciona el contexto necesario para tomar decisiones informadas sobre qué excepciones capturar y cómo manejarlas.

En Python, todas las excepciones heredan de la clase base `BaseException`.

Jerarquía principal:

- `BaseException`
  - `Exception`
    - `ValueError`
    - `TypeError`
    - `KeyError`
    - `IndexError`

    entre otras.

Organizar las excepciones en esta estructura permite manejar errores de forma específica o general según sea necesario.

¿Por qué usar excepciones específicas?

Elegir, manejar excepciones específicas no solo mejora la calidad del código, sino también facilita su comprensión y mantenimiento. Esta sección explora las razones clave para optar por esta práctica.

Claridad y depuración

Facilita identificar qué tipo de error ocurrió y dónde.

Ayuda a documentar intenciones claras en el manejo de errores.

Precisión en la lógica de recuperación

Permite tomar decisiones basadas en el tipo de problema.

Mejora la robustez del código al evitar suposiciones incorrectas.

Elección de la excepción correcta

Cada tipo de excepción en Python tiene un propósito específico. Esta sección detalla las diferencias entre las excepciones más comunes y cómo elegir la adecuada para cada situación.

`ValueError` : Cuando un argumento tiene el tipo correcto, pero un valor inapropiado

```
try:
    int("abc")
except ValueError as e:
    print(f"Error [ValueError]: No se pudo convertir el valor 'abc' a un entero. Detall
```

Copiar

`TypeError`: Cuando se realiza una operación con un tipo de dato incorrecto.

```python
try:
    "2" + 2
except TypeError as e:
    print(f"Error [TypeError]: Intentaste combinar un string con un número. Detalles: {
```

Copiar

`KeyError`: Al intentar acceder a una clave inexistente en un diccionario.

```python
try:
    my_dict = {"a": 1}
    value = my_dict["b"]
except KeyError as e:
    print(f"Error [KeyError]: La clave 'b' no existe en el diccionario. Detalles: {e}")
```

Copiar

`IndexError`: Cuando se intenta acceder a un índice fuera del rango de una lista.

```python
try:
    my_list = [1, 2, 3]
    value = my_list[10]
except IndexError as e:
    print(f"Error [IndexError]: El índice 10 está fuera del rango de la lista. Detalles
```

Copiar

`AttributeError`: Cuando un objeto no tiene un atributo solicitado.

```python
try:
    obj = 10
    obj.some_method()
except AttributeError as e:
    print(f"Error [AttributeError]: El objeto de tipo 'int' no tiene el atributo 'some_
```

Copiar

`ZeroDivisionError`: Cuando se intenta dividir un número por cero.

```python
try:
    result = 10 / 0
except ZeroDivisionError as e:
    print(f"Error [ZeroDivisionError]: Intentaste dividir 10 entre 0. Detalles: {e}")
```

Copiar

Manejo Personalizado de Excepciones

En situaciones complejas, es útil definir excepciones personalizadas para manejar errores específicos relacionados con el dominio del problema. Esto mejora la legibilidad y proporciona un manejo más intuitivo de los errores.

Ejemplo:

Imagina un sistema bancario donde debes validar transacciones. Puedes crear una excepción personalizada para detectar errores específicos como fondos insuficientes.

```python
class InsufficientFundsError(Exception):
    def __init__(self, balance, amount):
        super().__init__(f"Fondos insuficientes: Intentaste retirar {amount}, pero solo t

try:
    balance = 100
    amount_to_withdraw = 150
    if amount_to_withdraw > balance:
        raise InsufficientFundsError(balance, amount_to_withdraw)
except InsufficientFundsError as e:
    print(f"Error detectado: {e}")
```

Copiar

Cuándo usarlas:

Cuando se necesita una semántica más clara para un contexto particular.

Cuando se desea encapsular una lógica de error específica.

Beneficios:

Mejora la legibilidad y el mantenimiento del código.

Ayuda a manejar errores propios del dominio del problema, haciendo que el código sea más comprensible y mantenible.

Patrones y Antipatrones

Un manejo efectivo de excepciones requiere seguir buenas prácticas y evitar errores comunes. Esta sección describe patrones recomendados y antipatrones que deben evitarse.

Patrones

Capturar solo las excepciones esperadas

```python
try:
    resultado = 10 / divisor
except ZeroDivisionError:
    print("El divisor no puede ser cero.")
```

Copiar

Antipatrones

Capturar todo con `Exception` sin discriminación

```python
try:
    resultado = 10 / divisor
except Exception:
    print("Algo salió mal.")  # Difícil de depurar y entender.
```

Copiar

Problemas

Oculta la causa real del error:

Cuando se captura una excepción genérica, el mensaje puede no reflejar el problema exacto. Esto complica entender qué sucedió realmente, ya que no se proporciona información sobre el tipo de error o su contexto.

Dificulta el mantenimiento y la depuración.

Al capturar todas las excepciones indiscriminadamente, el flujo del programa se vuelve menos predecible. Esto puede generar errores secundarios que son más difíciles de rastrear.

Por ejemplo, un error inesperado podría ser silenciado, causando que el programa continúe en un estado inconsistente.

Capturar excepciones específicas permite identificar y resolver problemas de manera más eficiente y precisa.

Conclusión

El manejo adecuado de excepciones es una habilidad esencial en la programación. Este cierre resume los aprendizajes clave y destaca cómo implementarlos en futuros proyectos.

Recapitulación:

1. **Claridad en el manejo de errores:** Usar excepciones específicas permite identificar de manera rápida y precisa los problemas que ocurren durante la ejecución.
2. **Facilidad de mantenimiento:** Un código que utiliza excepciones correctamente es más fácil de depurar y mantener, lo que contribuye a la calidad a largo plazo del software.
3. **Evitación de errores comunes:** Capturar solo las excepciones esperadas y evitar antipatrones mejora significativamente la previsibilidad y robustez del programa.

Recomendaciones prácticas:

Evalúa cuidadosamente qué excepciones podrían surgir en cada bloque de código.

Documenta el propósito y alcance de las excepciones personalizadas para mejorar la comprensión del equipo de desarrollo.

Revisa el manejo de excepciones periódicamente para adaptarlo a nuevas necesidades o casos de uso.